Supporting Information for Murray, Patel, and Wiedenmann (2022)

1 Unity rendering tests

1.1 Creating the project

The format of Unity project files changes quickly, so instead of providing a project file as Supporting Information, we describe how to create the project described in the main article. We developed the following procedure in Unity 2020.1.5f1.

(a) Main Camera. Create a new Unity project using the '3D' template. At the time of writing, every new project contains a camera called 'Main Camera'. If the project does not have this camera, create one, and name it 'Main Camera'. (Object names are important for the script that will run the rendering tests, and are case-sensitive.) In the camera's Transform component, set the position to (0, 0, -10) and the rotation to (0, 0, 0). This places the camera on the negative z-axis, facing the origin.

(b) Directional Light. The new project should also contain a directional light called 'Directional Light'. If the project does not have this object, create it, and name it 'Directional Light'. Set the position to (0, 0, -20) and the rotation to (0, 0, 0). This places the light on the negative z-axis, directing light towards the origin. Assign it the RGB color (255, 255, 255). Also turn off environment lighting (i.e., all light sources other than the Directional Light).

(c) Plane. Create a GameObject of type Plane, and name it 'Plane'. Set the position to (0, 0, 0), the rotation to (-90, 0, 0), and the scale to (0.5, 0.5, 0.5). This places the plane at the origin, facing the camera.

(d) Materials. Create a new material of type Standard, and name it 'standard'. Also create a new material of type Legacy Shaders/Diffuse, and name it 'lambertian'. Assign the Lambertian material to the Plane object, by dragging and dropping it from the Assets folder onto the Plane object in the Hierarchy window.

(e) Script. The Supporting Information includes a C# script named `MainScript.cs`. Add this script to the project's Assets folder. Attach it to the Main Camera object, by dragging and dropping it from the Assets folder onto the Main Camera object in the Hierarchy window.

(f) Post-processing. Install the Post Processing Stack package (version 3.2) into the project. Installation instructions are available at `https://docs.unity3d.com/Packages/com.unity.postprocessing@3.2/manual/index.html`, under 'Installation'. The instructions outlined in the following paragraphs for configuring the post-processing stack are also available at the same URL, under 'Getting Started'.

After installing the package, add a Post-process Layer component to the Main Camera object. To do this, select the Main Camera in the Hierarchy window, and the in the Inspector view, press Add Component, and choose Rendering/Post-Process Layer. Still in the Inspector view, set the Layer property of the component to Everything in the drop-down menu.

Next create an empty GameObject and call it PostProcess. Add a Post-Process Volume component to this new object. To do this, select the PostProcess object in the Hierarchy window, and in the Inspector View, press Add Component, and choose Rendering/Post-Process Volume. In the Inspector view, check the checkbox of the Is Global property of this component; also

press the New button of the Profile property.

Next add a Color Grading effect to the Post-Process Volume component. To do this, in the Inspector view, at the bottom of the Post-Process Volume component, press the 'Add effect...' button, and choose Unity/Color Grading. To see the color grading options, click the small triangle at the top of the box that is added for this effect. At the top of the larger box that opens, check the checkbox of the Mode property, and set its value to Low Definition Range from the drop-down menu; also check the checkbox of the Lookup Texture property.

To load a LUT texture into the color grading component, first add the LUT texture to the Assets folder. Then choose the texture as the active LUT via the selection box to the right of the Lookup Texture property. You may see a warning message, 'Invalid LUT import settings'; click the Fix button next to this message. The LUT texture will now modify the rendered RGB values as described in the main paper.

This step has described how to configure the post-processing stack and color grading component for a standard Main Camera Object. The same steps can be used to use a LUT texture with any other type of virtual camera as well, including the VR camera component. Simply add the post-process layer component described above to the VR camera component, instead of the Main Camera component.

(g) Testing the result. To test whether the post-processing stack is configured correctly, load the LUT texture `lut_red.png` into the color grading component, as described in step (f) above. This LUT leaves the red channel unchanged, and sets the green and blue channels to zero. If the post-processing stack is configured correctly then the entire scene should be shown in red in the Game view and when running the project.

1.2 Running the tests

After this project is created, the rendering tests reported in the main article can be replicated. In `MainScript.cs`, at line 11, a variable `test` is declared that controls which test is carried out when the project runs. With `test=1`, the Plane object is rotated about the vertical axis, from -85° to 85°. At each angle, the RGB value rendered at the centre of the plane is recorded. The rotation angles and RGB values are written to a text file named `data.txt` in the project folder. The first column is the angle, and the remaining three columns are the RGB values. Figure 2 in the main article is based on this data. For Figure 2a, the user must manually assign the Lambertian material to the Plane object, set the Gamma/Linear property to Gamma, and disable the PostProcessing game object. The red data points in Figure 2a show the rendered RGB value versus the angle of the Plane object. Figure 2b is created the same way, except that the Gamma/Linear property must be set to Linear. The project does not create the plots, but `data.txt` is a CSV file that can be loaded into a data processing program such as MATLAB or R.

With `test=2`, the Plane object is assigned achromatic RGB values from 0 to 255. For each assigned RGB value, the RGB value rendered at the centre of the plane is recorded. The assigned and rendered RGB values are written to `data.txt`. The first column is the assigned achromatic RGB value, and the remaining three columns are the rendered RGB values. To generate the data for Figure 2c, assign the Lambertian material to the Plane object, and select either Gamma or Linear mode, and disable the PostProcessing game object. Figure 2c plots the rendered versus assigned RGB values in Gamma and Linear

modes. To generate the data for Figure 4, repeat the experiment with the PostProcessing game object enabled, and the LUT textures `lut_identity.png` or `lut_triangle.png` loaded into the Lookup Table property of the color grading effect in the Post-Processing Volume.

None of the tests described here use the Standard material. To see that this material is not Lambertian with its default parameter settings (as mentioned in endnote 1 in the main article), assign it to the Plane object and run the project with `test=1` in Gamma mode. Note that the RGB values recorded in `data.txt` are not a cosinusoidal function of the angle of the Plane object relative to the light source.

This project and code can be modified to carry out many other rendering tests as well.

2 Creating LUT textures

The Supporting Information provides MATLAB code to create LUT textures for luminance calibration. The function `unitylut.m` takes calibration data (measurements of luminance for several achromatic RGB values), and produces a LUT texture image file (.PNG format) that, when loaded into the Post-Processing Volume (see previous section), linearizes the relationship between achromatic RGB value and luminance. The script `unitylut_demo.m` illustrates how to use this function.

The function `unitylut.m` relies on a model of color grading where RGB values are linearly interpolated between the values specified in the LUT texture. As noted in the main article, Unity's color grading sometimes has small departures from linear interpolation, so this is an approximation.